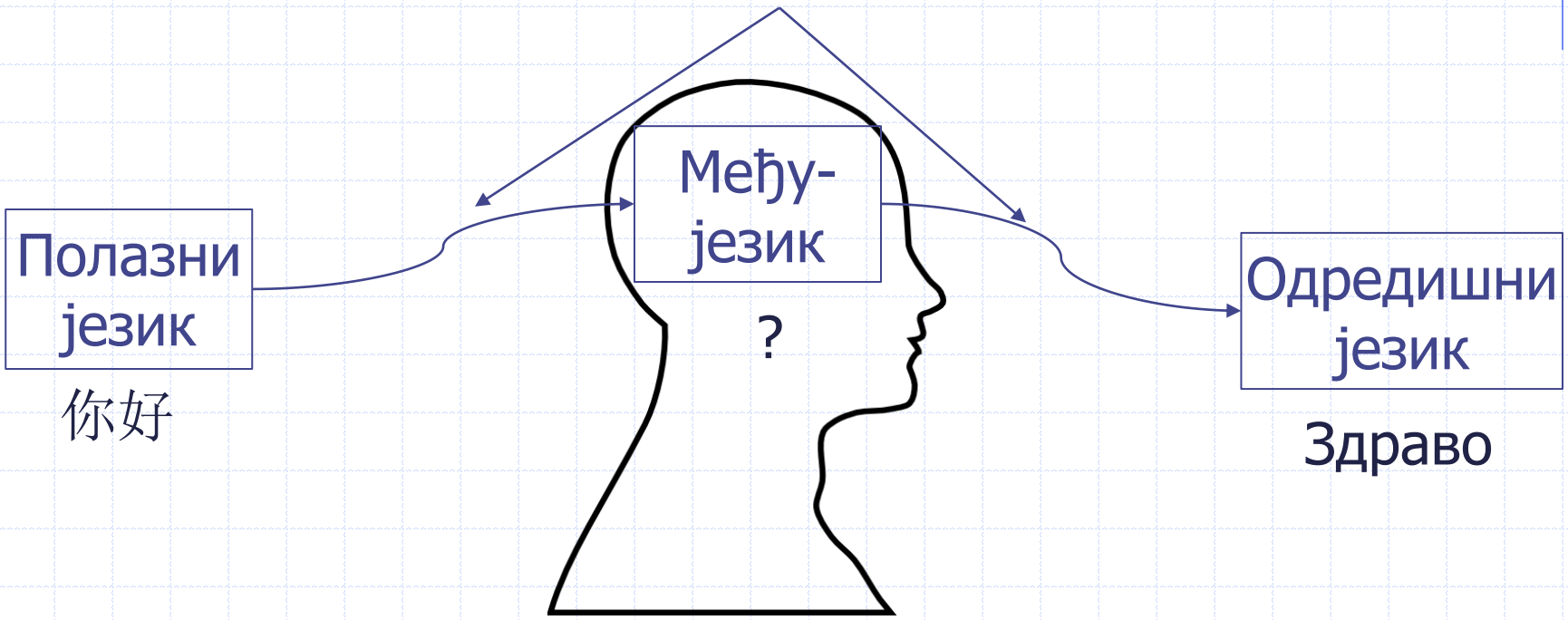
Међујезик/Међурепрезентација

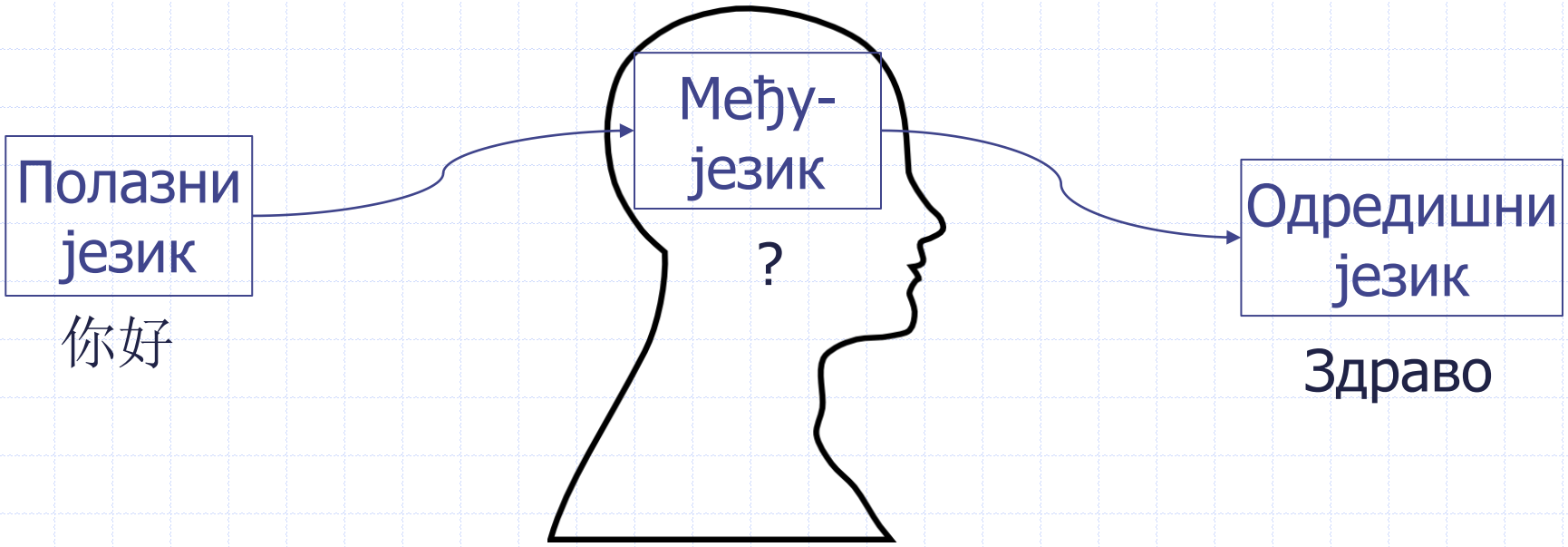
- Појам међујезика
- Синтакса међујезика
- Представа међујезика

Превођење

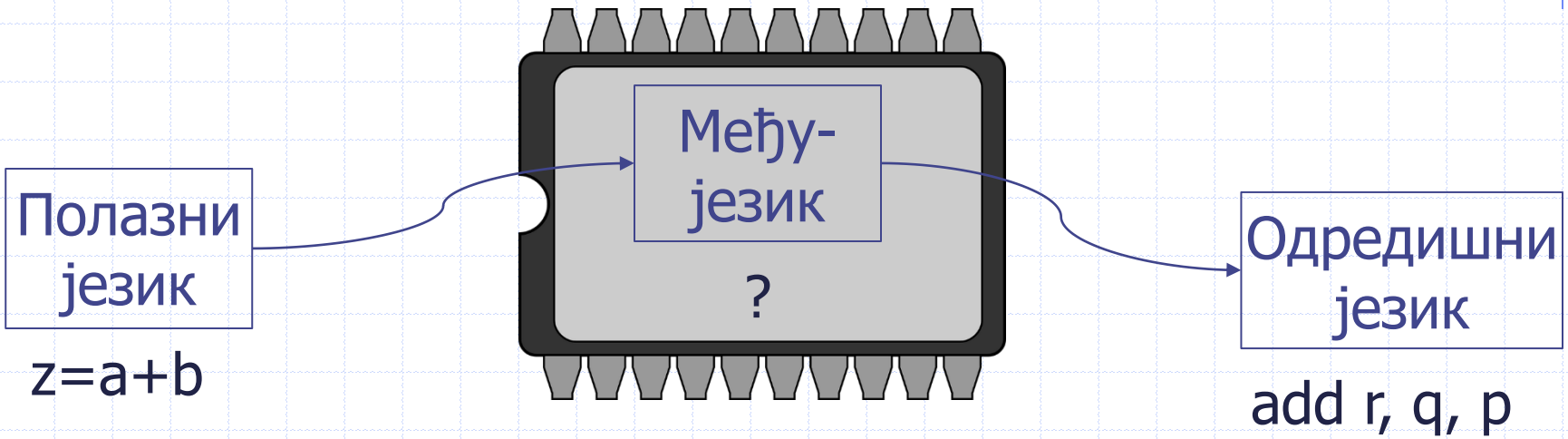
Неки медијум: текст (на папиру, датотека у рачунару...),
говор (уживо, преко телефона, са плоче...), ...



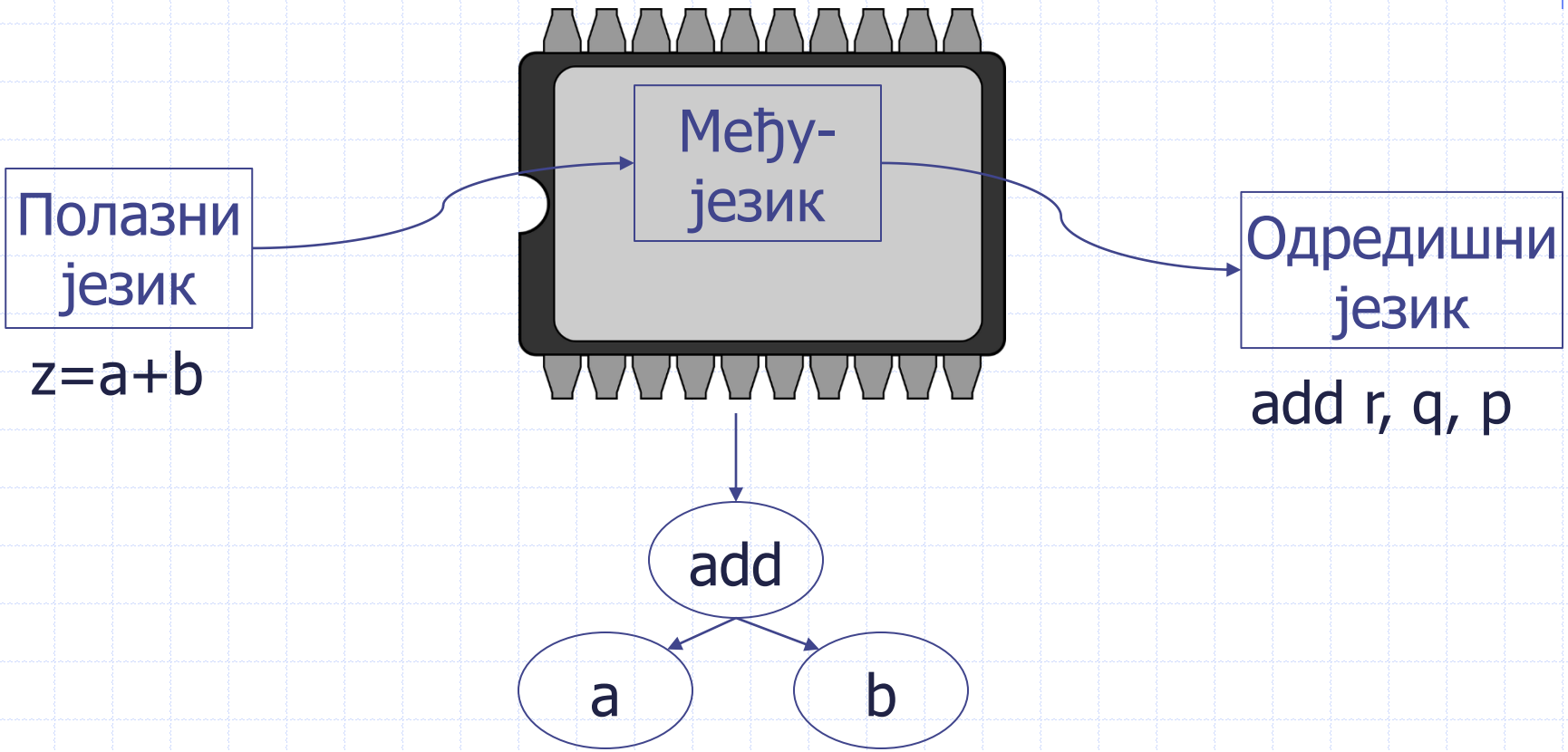
Превођење



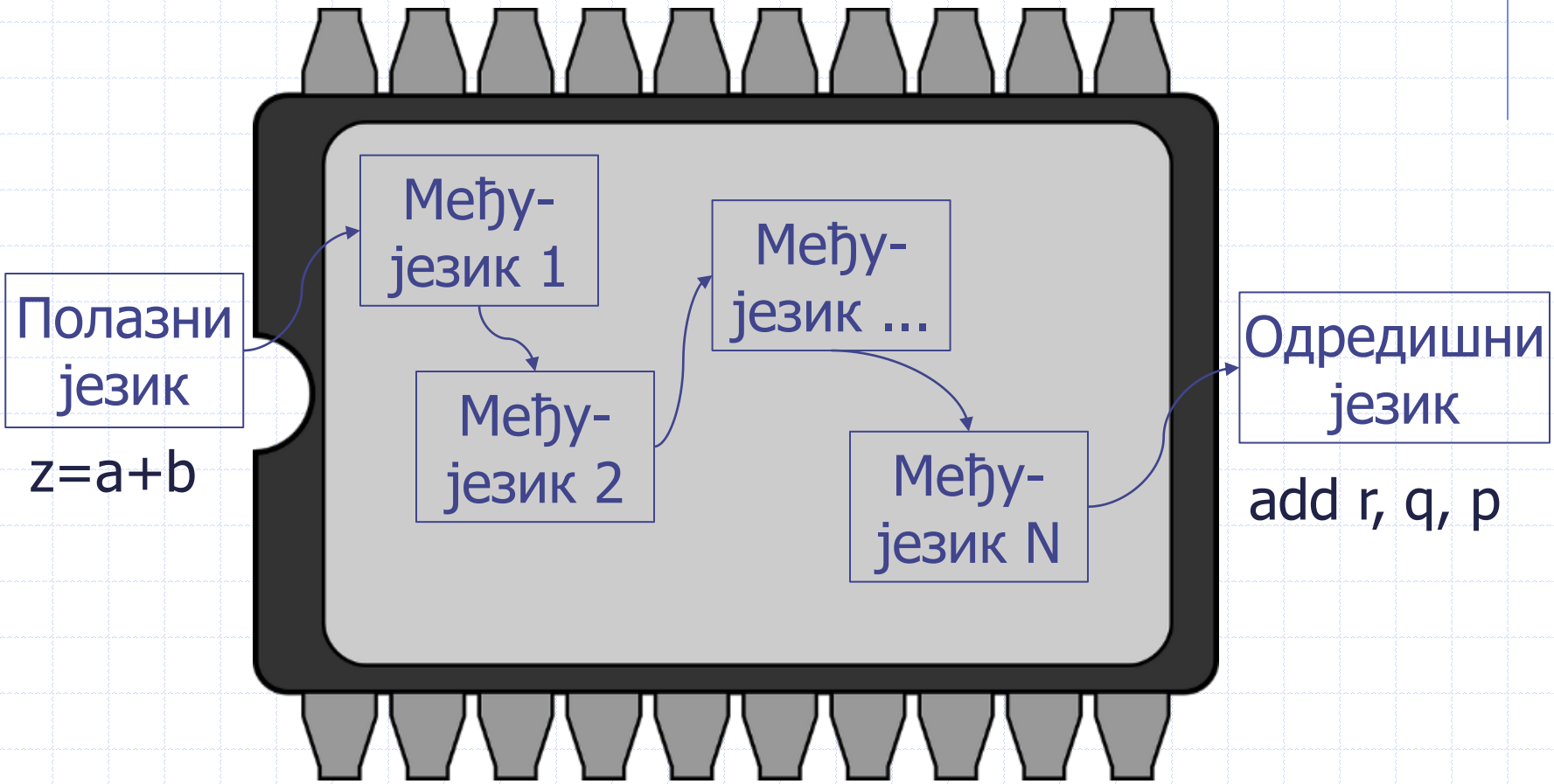
Превођење



Превођење



Превођење


Полазни језик
z=a+b
Међу-језик 1
Међу-језик 2
Међу-језик ...
Међу-језик N
Одредишни језик
add r, q, p

Међујезик (међу-репрезентација)

- Компајлер користи прелазне представе програма који се преводи
  - Међујезици
    - Ове представе су најчешће у облику стабала са неколико врста чворова
    - Свака врста чвора има своје атрибуте
  - Представе у облику стабала се могу описати граматикама, баш као и програмски језици
    - Пример: праволинијски програмски језик, са исказима и изразима, без петљи или иф исказа
    - Тзв. језик праволинијских програма
    - A.W.Apple, Modern compiler implementation in C/Java/ML

Синтакса праволинијског језика (1/2)

Stm → ***Stm*** ; ***Stm***	(CompoundStm)	***ExpList*** → ***Exp*** , ***ExpList***	(PairExpList)
Stm → id := ***Exp***	(AssignStm)	***ExpList*** → ***Exp***	(LastExpList)
Stm → print (***ExpList***)	(Printstm)	***Binop*** → +	(Plus)
Exp → id	(IdExp)	***Binop*** → -	(Minus)
Exp → num	(NumExp)	***Binop*** → x	(Times)
Exp → ***Exp Binop Exp***	(OpExp)	***Binop*** → /	(Div)
Exp → (***Stm*** , ***Exp***)	(EseqExp)		

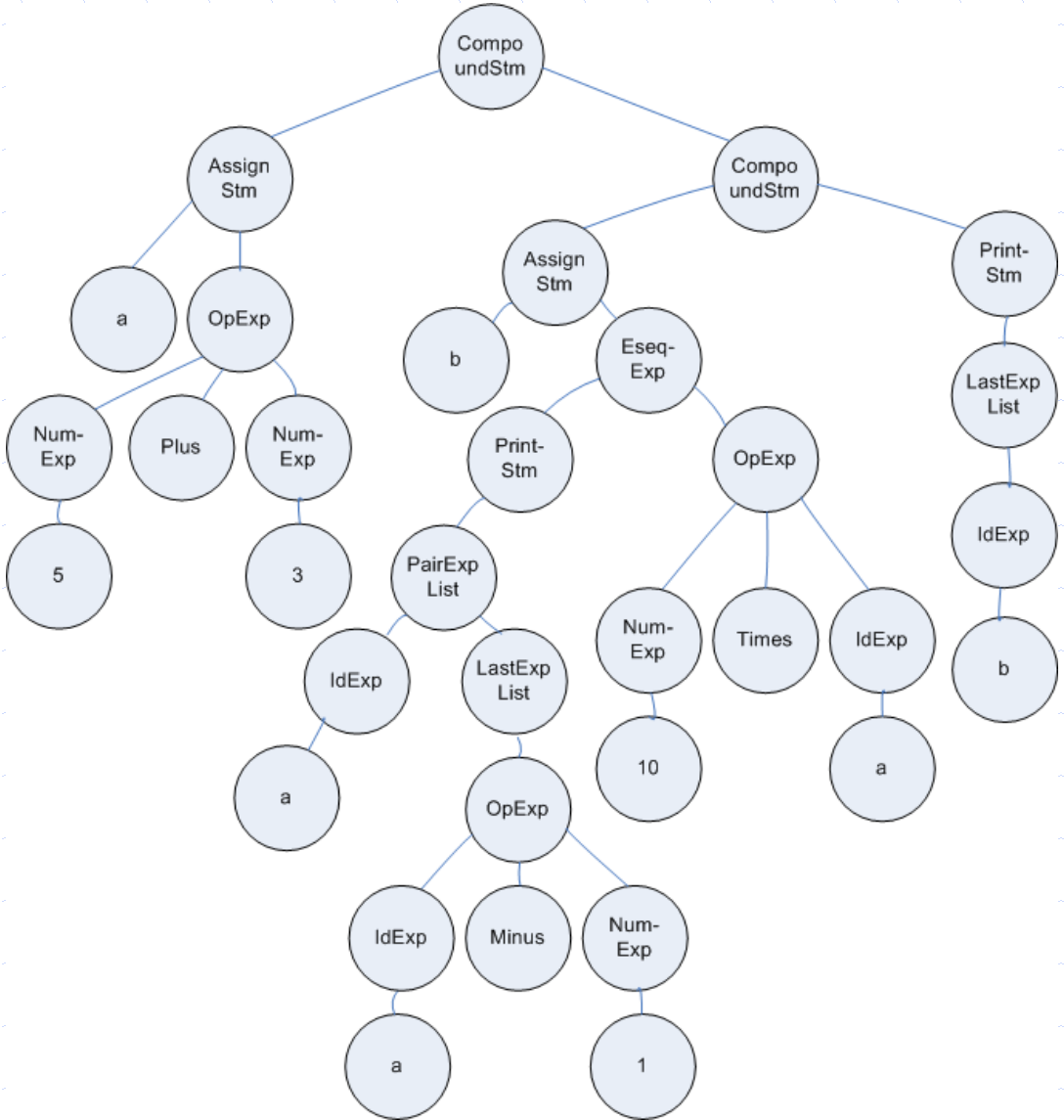
- Сваки ***Stm*** је исказ. Сваки ***Exp*** је израз.
- Исказ *s1* ; *s2* извршава најпре *s1*, затим *s2*.
- Исказ *i* := *e* израчунава израз *e*, и његову вредност смешта у променљиву *i*.
- print(*e1*, *e2*, ... , *en*) приказује вредности израза, израчунатих с лева у десно, раздвојено празним местима и преласком у нову линију на крају.

Синтакса праволинијског језика (2/2)

- Вредност *израза са идентификатором*
  - је вредност идентификатора; нпр. за израз *i* је то вредност у *i*
- Израз са бројем (num) се своди на вредност тог целог броја
- Израз са оператором *e1* op *e2*
  - Израчунава се *e1*, затим *e2*, и примењује дати оператор op
- Низ израза (*s*, *e*) се понаша слично оператору „,“ језика Це
  - Извршава се исказ *s* због бочних ефеката (уписа у променљиве), затим се израчунава и враћа вредност израза *e*

Пример праволинијског програма

a := 5 + 3; b := (print (a , a – 1) , 10 * a) ; print (b)



Представа програма

- Излаз програма: 8 7 CR LF 80
- Како представити овакав програм?
  - Један начин је у облику низа знакова (изворни код)
  - Погоднији начин (за преводиоца) је у облику структуре података у облику стабла, са по једним чвором за сваки исказ (Stm) и израз (Exp)
- Представа програма у облику стабла
  - Чворови су означени именима продукција полазне граматике
  - Сваки чвор има онолико чворова потомака, колико има симбола с десне стране продукције
  - Могуће је директно превести граматику у структуру података
    - Сваки граматички симбол (синтаксна класа) одговара једном типу у структури података

Превођење граматике у структуру података у Це++-у

Граматички симбол	Тип
Stm	IrStm
Exp	IrExp
ExpList	IrExpList
id	string
num	int

- Свако граматичко правило
  - има конструктор за симбол с леве стране
  - има компоненте с десне стране, које треба представити
    - CompoundStm има 2 IrStm-а, AssignStm има id и IrExp, итд.

Превођење граматике у структуру података у Це++-у

- Постоји неколико начина да се оваква структура опише у Це++-у.
- У књизи је приказан Це-овски начин, ослањајући се на уније.
- Овде на слајдовима је приказан ОО начин који се ослања на хијерархију класа.
- Осим ова два, у материјалима уз ово предавање илустровано је још неколико начина.

Идеја

- Стабла су описана граматиком
- Симболима у граматици се придружују типови (класе)
- Сваки тип садржи
  - поље kind, које је енум различитих варијанти, једне по сваком правилу
- Сваки тип има конструктор који поставља сва поља
- Сваки тип има конструкциону функцију (make*)

Представа праволинијских програма (1/5): Објектни стил – дефиниције базних класа

```
enum StmKind { SK_COMPOUND, SK_ASSIGN, SK_PRINT };
enum ExpKind { EK_ID, EK_NUM, EK_OP, EK_ESEQ };
enum ExpListKind { ELK_PAIR, ELK_LAST };

struct IrStm {
    StmKind kind;
protected:
    Stm(StmKind k) : kind(k) {}
};

struct IrExp {
    ExpKind kind;
protected:
    IrExp(ExpKind x) : kind(x) {}
};

struct IrExpList {
    ExpListKind kind;
protected:
    IrExpList(ExpListKind x) : kind(x) {}
};
```

Представа праволинијских програма (2/5): Објектни стил – класе исказа

```
struct CompoundStm : IrStm{
    CompoundStm(IrStm* s1, IrStm* s2)
        : Stm(SK_COMPOUND), stm1(s1), stm2(s2) {}
    IrStm* stm1;
    IrStm* stm2;
};

struct AssignStm : IrStm {
    AssignStm(std::string x, IrExp* y)
        : IrStm(SK_ASSIGN), id(x), exp(y) {}
    std::string id;
    IrExp* exp;
};

struct PrintStm : IrStm {
    PrintStm(IrExpList* x) : IrStm(SK_PRINT), exps(x) {}
    IrExpList* exps;
};
```

Stm → ***Stm*** ; ***Stm***

Stm → **id** := ***Exp***

Stm → print (***ExpList***)

Представа праволинијских програма (3/5): Објектни стил – класе израза

```
enum OpKind { OK_PLUS, OK_MINUS, OK_MUL, OK_DIV };

struct IdExp : IrExp {
    IdExp(const std::string& x) : IrExp(EK_ID), name(x) {}
    std::string name;
};

struct NumExp : IrExp {
    NumExp(int x) : IrExp(EK_NUM), value(x) {}
    int value;
};

struct OpExp : IrExp {
    OpExp(IrExp* x, OpKind op, IrExp* y)
        : IrExp(EK_OP), left(x), operKind(op), right(y) {}
    OpKind operKind;
    IrExp* left;
    IrExp* right;
};

struct EseqExp : IrExp {
    EseqExp(IrStm* x, IrExp* y) : IrExp(EK_ESEQ), stm(x), exp(y) {}
    IrStm* stm;
    IrExp* exp;
};
```

Exp → id

Exp → num

Exp → *Exp Binop Exp*

Exp → (*Stm* , *Exp*)

Представа праволинијских програма (4/5): Објектни стил – листа исказа

```
struct PairExpList : IrExpList {
    PairExpList(IrExp* x, IrExpList* y)
        : IrExpList(ELK_PAIR), head(x), tail(y) {}
    IrExp* head;
    IrExpList* tail;
};
```

ExpList → *Exp* , *ExpList*

ExpList → *Exp*

```
struct LastExpList : IrExpList {
    LastExpList(IrExp* x) : IrExpList(ELK_LAST), exp(x) {}
    IrExp* exp;
};
```

Представа праволинијских програма (5/5): Објектни стил – конструкционе функције

```
IrStm* makeCompoundStm(IrStm* x, IrStm* y) { return new CompoundStm(x, y); }
IrStm* makeAssignStm(std::string x, IrExp* y) { return new AssignStm(x, y); }
IrStm* makePrintStm(IrExpList* x) { return new PrintStm(x); }


IrExp* makeIdExp(const std::string& x) { return new IdExp(x); }
IrExp* makeNumExp(int x) { return new NumExp(x); }
IrExp* makeOpExp(IrExp* x, OpKind op, IrExp* y) {
    return new OpExp(x, op, y);
}
IrExp* makeEseqExp(IrStm* x, IrExp* y) { return new EseqExp(x, y); }


IrExpList* makePairExpList(IrExp* x, IrExpList* y) {
    return new PairExpList(x, y);
}
IrExpList* makeLastExpList(IrExp* x) { return new LastExpList(x); }
```